

TP n° 1

Société Orion

*Conception et implémentation
d'un Data Warehouse opérationnel*

Modélisation opérationnelle 3NF, transition vers la modélisation dimensionnelle en étoile, génération de données fictives via Faker, ETL implémenté en *procédures stockées T-SQL* sur *SQL Server* et orchestré depuis Python, déploiement Docker complet (Postgres OLTP + Postgres DWH + SQL Server), réponses analytiques aux 15 questions du cahier des charges.

Table des matières

1	Contexte et objectifs	3
1.1	La société Orion	3
1.2	Objectifs décisionnels	3
1.3	Démarche retenue	3
2	Modélisation opérationnelle (OLTP)	4
2.1	Principes	4
2.2	Carte des sous-domaines	4
2.3	Sous-domaine Géographie & Organisation	4
2.4	Sous-domaine Produits & Fournisseurs	4
2.5	Sous-domaine Clients & Commandes	4
2.6	Choix structurants	5
2.7	Listing : extrait du DDL OLTP	5
3	Transition OLTP → modélisation dimensionnelle	6
3.1	Étape 1 — Identifier le processus métier	6
3.2	Étape 2 — Choisir le grain	6
3.3	Étape 3 — Identifier les dimensions	6
3.4	Étape 4 — Identifier les faits	7
3.5	Bus matrix	7
3.6	Vue synthétique de la transition	8
4	Modélisation dimensionnelle (DWH)	9
4.1	Étoile vs flocon	9
4.2	Schéma en étoile	9
4.3	Stratégie SCD (Slowly Changing Dimensions)	9
4.4	Mesures de la fact	9
5	Architecture technique : stack Docker	10
5.1	Vue d'ensemble	10
5.2	docker-compose.yml (extrait)	10
5.3	Configuration et démarrage	11
6	Génération des données opérationnelles	12
6.1	Stratégie	12
6.2	Réalisme injecté	12
6.3	Listing : extrait du générateur	12
7	ETL T-SQL sur SQL Server	13
7.1	Architecture en deux niveaux	13
7.2	Procédures stockées T-SQL	13
7.3	Cur SCD2 en T-SQL	13
7.4	Watermark et idempotence	14
7.5	Orchestrateur Python	14
7.6	Journalisation	14
7.7	Variante « tout SQL Server »	14
8	Réponses aux questions analytiques	15
Q1.	Quels sont les produits qui se vendent le mieux ?	15
Q2.	Quels sont les produits en perte de vitesse ?	15
Q3.	Produits qui contribuent très peu au CA pour un pays / une année donnés ?	15
Q4.	Marge générée par un groupe de produits, par année	16

Q5. La marge dépend-elle de la quantité vendue ?	16
Q6. Les remises font-elles augmenter les ventes (Q6) et la marge (Q7) ?	16
Q7. Commerciaux qui font le plus de ventes	16
Q8. Commerciaux les plus performants par pays / sexe / âge / salaire	17
Q9. Quels groupes de clients sont identifiés ?	17
Q10. Clients les plus rentables (top 50)	17
Q11. Fournisseurs proposant des produits rentables	17
Q12. Moyenne et écart-type du chiffre d'affaires	18
Q13. Variables qui expliquent le mieux le chiffre d'affaires	18
Q14. Différence significative de CA entre commerciaux femmes et hommes ?	19
Annexe A — Arborescence du projet	20
Annexe B — Démarrage et exploitation	20

1 Contexte et objectifs

1.1 La société Orion

Orion est une société fictive mondiale, spécialisée dans la commercialisation d'articles de sport et d'extérieur. Son siège social est aux États-Unis et elle gère des filiales en Belgique, Pays-Bas, Allemagne, Royaume-Uni, Danemark, France, Italie, Espagne et Australie.

Les ventes se font via trois canaux : magasins physiques, catalogue et internet. Le programme de fidélité *Orion Star Club* confère des avantages aux clients qui en sont membres. L'historique disponible va du **1^{er} janvier 1998 au 31 décembre 2002**.

Volumétrie cible.

- ~ 5 500 références produit
- ~ 90 000 clients (pas tous actifs)
- ~ 980 000 commandes
- ~ 600 à 800 employés
- 64 fournisseurs (un seul fournisseur par produit)

1.2 Objectifs décisionnels

Les questions auxquelles le système doit répondre se regroupent en cinq familles :

Performance produit

best-sellers, perte de vitesse, contribution au chiffre d'affaires, marges par groupe.

Effet remise

impact des remises sur les ventes et la marge.

Performance commerciale

commerciaux les plus performants par pays, sexe, âge, salaire.

Segmentation client

groupes de clients, clients les plus rentables.

Analyses statistiques

moyennes et écarts-types du chiffre d'affaires, variables explicatives, comparaison hommes/-femmes parmi les commerciaux.

1.3 Démarche retenue

La construction de l'entrepôt a été décomposée en sept étapes :

1. Modélisation **opérationnelle** (3NF) du système source — section 2.
2. **Transition** OLTP → modélisation dimensionnelle (méthode Kimball en 4 étapes) — section 3.
3. Modélisation **dimensionnelle** (étoile) de l'entrepôt — section 4.
4. Provisionnement de l'environnement via **Docker Compose** (Postgres OLTP + Postgres DWH + SQL Server) — section 5.
5. Génération de jeux de données réalistes via **Faker** — section 6.
6. **ETL T-SQL sur SQL Server** et son orchestration Python — section 7.
7. Rédaction des **requêtes analytiques** répondant aux questions métier — section 8.

2 Modélisation opérationnelle (OLTP)

2.1 Principes

L'objectif du modèle opérationnel est de soutenir les opérations courantes : prise de commande, gestion des stocks, gestion RH, suivi des fournisseurs. Les contraintes fortes y sont l'*intégrité référentielle* et la *non-redondance*. Le modèle est en troisième forme normale (3NF) et range toutes les tables dans un schéma PostgreSQL nommé `ops`.

2.2 Carte des sous-domaines

Le système se décompose en quatre sous-domaines fortement couplés via la fact transactionnelle (les commandes).

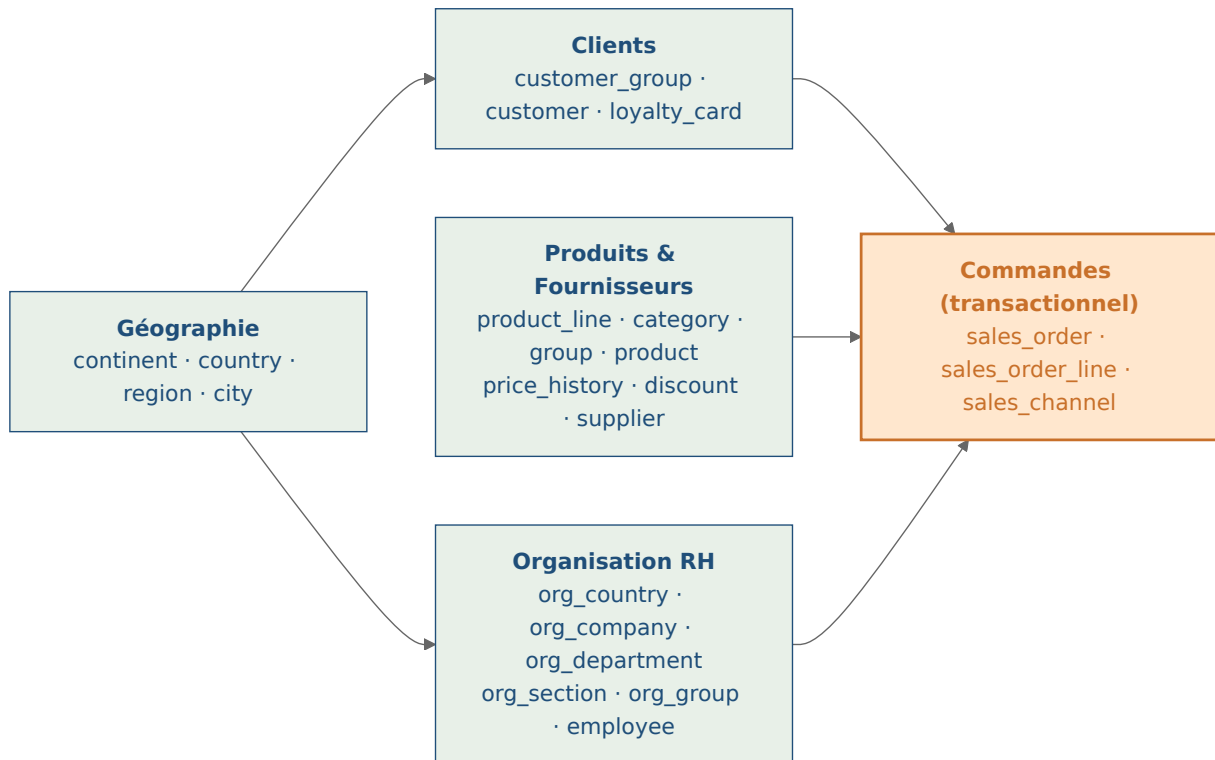


Figure 1 – Carte de domaines : les commandes constituent la fact transactionnelle alimentée par les trois domaines référentiels.

2.3 Sous-domaine Géographie & Organisation

Deux hiérarchies parallèles cohabitent : la géographie commerciale (utilisée par les fournisseurs et les clients) et la hiérarchie organisationnelle des employés (5 niveaux comme imposé par l'énoncé).

2.4 Sous-domaine Produits & Fournisseurs

La hiérarchie produit comporte les 4 niveaux requis. L'historique de prix et les remises sont externalisés dans deux tables distinctes pour conserver le grain temporel.

2.5 Sous-domaine Clients & Commandes

C'est le cœur transactionnel. La carte de fidélité *Orion Star Club* est externalisée pour ne pas polluer la table `client` (relation 1-1 optionnelle).

2.6 Choix structurants

- **Hiérarchie produit** (4 niveaux) et **hiérarchie organisationnelle** (5 niveaux) sont *normalisées* : une table par niveau, FK « enfant vers parent ». Cela évite la duplication des libellés et simplifie les renommages.
- **Historique de prix** dans `product_price_history` : on retrouve le prix appliqué au moment de la commande en sélectionnant la fenêtre (`start_date`, `end_date`) contenant `order_date`. La fenêtre courante a `end_date` IS NULL.
- **Remises** : `product_discount` avec validité bornée, permettant les promotions saisonnières.
- **Auto-référence** `employee.manager_id` pour la hiérarchie de management (n+1).
- **Carte de fidélité** : table dédiée plutôt qu'attribut booléen, pour stocker numéro de carte et date d'émission.
- Tous les montants sont en **USD** (cf. énoncé).

2.7 Listing : extrait du DDL OLTP

L'intégralité du schéma est dans `oltp/init/01_schema.sql`. Voici la table centrale des commandes et l'historique de prix.

Listing 1 – Tables de commandes (extrait)

```
1 CREATE TABLE commande (
2     commande_id    BIGSERIAL PRIMARY KEY,
3     date_commande  DATE NOT NULL,
4     client_id      BIGINT  NOT NULL REFERENCES client(client_id),
5     employe_id     BIGINT  NOT NULL REFERENCES employe(employe_id),
6     canal_id       SMALLINT NOT NULL REFERENCES canal_vente(canal_id),
7     montant_total  NUMERIC(14,2) NOT NULL DEFAULT 0,
8     cree_le        TIMESTAMP NOT NULL DEFAULT now()
9 );
10
11 CREATE TABLE ligne_commande (
12     commande_id    BIGINT  NOT NULL REFERENCES commande(commande_id) ON DELETE CASCADE,
13     numero_ligne    SMALLINT NOT NULL,
14     produit_id      BIGINT  NOT NULL REFERENCES produit(produit_id),
15     quantite        NUMERIC(12,3) NOT NULL CHECK (quantite > 0),
16     prix_unitaire   NUMERIC(12,2) NOT NULL CHECK (prix_unitaire >= 0),
17     cout_unitaire   NUMERIC(12,2) NOT NULL CHECK (cout_unitaire >= 0),
18     pct_remise      NUMERIC(5,4) NOT NULL DEFAULT 0
19                     CHECK (pct_remise BETWEEN 0 AND 1),
20     PRIMARY KEY (commande_id, numero_ligne)
21 );
```

Listing 2 – Historique de prix avec validité ouverte

```
1 CREATE TABLE historique_prix (
2     prix_id        BIGSERIAL PRIMARY KEY,
3     produit_id     BIGINT  NOT NULL REFERENCES produit(produit_id) ON DELETE CASCADE,
4     date_debut     DATE NOT NULL,
5     date_fin       DATE,
6     cout           NUMERIC(12,2) NOT NULL CHECK (cout >= 0),
7     prix_vente     NUMERIC(12,2) NOT NULL CHECK (prix_vente >= 0),
8     CHECK (date_fin IS NULL OR date_fin >= date_debut)
9 );
10 CREATE INDEX idx_historique_prix_periode
11     ON historique_prix(produit_id, date_debut, date_fin);
```

3 Transition OLTP → modélisation dimensionnelle

Cette section formalise *comment* on est passé du modèle 3NF à un schéma en étoile. La démarche suit la **méthode Kimball en quatre étapes** (Kimball & Ross, *The Data Warehouse Toolkit*, 3^e éd.).

3.1 Étape 1 — Identifier le processus métier

Le processus métier au cur des questions de l'énoncé (p. 3) est :

« Vendre un article à un client, via un canal donné, par un commercial donné, à une date donnée. »

C'est ce processus qui produit la mesure principale (chiffre d'affaires) et qui sera matérialisé par la table de faits.

3.2 Étape 2 — Choisir le grain

Choix structurant. Trois grains candidats ont été envisagés :

Grain candidat	Pour	Contre
Commande complète Ligne de commande ✓	Volume modeste Conserve le détail produit	Perd la dimension produit Volume = ~980 000 lignes (acceptable)
Ligne consolidée jour	Très petit volume	Perd commande, client précis, canal

Table 1 – Choix du grain : la **ligne de commande** est retenue.

Toutes les mesures de `fact_sales` seront additives au niveau ligne.

3.3 Étape 3 — Identifier les dimensions

À partir des questions analytiques, sept dimensions sont identifiées.

Question analytique (énoncé)	Dimension associée
« Quels produits se vendent le mieux ? »	<code>dim_produit</code>
« [...] par pays et année donnés »	<code>dim_geographie</code> + <code>dim_date</code>
« Marge par groupe de produit »	<code>dim_produit</code> (hiérarchie aplatie)
« Commerciaux par sexe, âge, salaire, pays »	<code>dim_employe</code> (avec attributs RH)
« Quels groupes de clients sont identifiés ? »	<code>dim_client</code> (groupe + démographie)
« Fournisseurs proposant des produits rentables »	<code>dim_fournisseur</code>
« Différence H/F entre les commerciaux »	<code>dim_employe.sexe</code>
Analyse par canal de vente	<code>dim_canal</code>

Table 2 – Dérivation des dimensions à partir des questions de l'énoncé.

Aplatissement des hiérarchies

Les hiérarchies normalisées de l'OLTP sont **aplaties** dans la dimension. Exemple produit : 4 tables OLTP → 1 dimension.

```
ops.ligne_produit
ops.categorie_produit      dim_produit (attributs aplatés)
ops.groupe_produit        - nom_produit
ops.produit                - nom_groupe_produit
                           - nom_categorie_produit
                           - nom_ligne_produit
                           - nom_fournisseur
```

Idem pour la géographie (4 tables → 1 dimension) et l'organisation RH (5 tables → attributs aplatis dans `dim_employee`). Cela permet :

- des **drill-down** rapides en SQL (`GROUP BY product_line_name, GROUP BY product_group_name, ...`);
- une **lisibilité** accrue pour les outils BI;
- un coût en redondance assumé — c'est précisément l'objet d'un DWH.

Surrogate keys

Chaque dimension porte une **clé technique** (surrogate key) générée à l'insertion : `cle_produit`, `cle_client`, `cle_employe`, ... La clé naturelle OLTP est conservée comme attribut (`produit_id`, `client_id`, ...) pour la traçabilité, mais **n'est jamais utilisée comme FK dans fait_ventes**.

Justifications :

- Découple le DWH des changements OLTP (renommage, fusion d'IDs).
- Indispensable pour SCD2 : plusieurs versions d'un client ont le *même* `client_id` mais des `cle_client` *différents*.

3.4 Étape 4 — Identifier les faits

Les mesures sont calculées à partir de la ligne de commande :

Mesure DWH	Source / formule	Type
quantite	<code>ligne_commande.quantite</code>	additive
prix_unitaire	<code>ligne_commande.prix_unitaire</code>	non-additive
cout_unitaire	<code>ligne_commande.cout_unitaire</code>	non-additive
pct_remise	<code>ligne_commande.pct_remise</code>	non-additive
montant_brut	<code>quantite × prix_unitaire</code>	additive
montant_remise	<code>brut × pct_remise</code>	additive
montant_net	<code>brut − remise</code>	additive (CA)
montant_cout	<code>quantite × cout_unitaire</code>	additive
montant_marge	<code>net − cout</code>	additive (marge)

Table 3 – Mesures de `fait_ventes` et leur additivité.

Pourquoi pré-calculer net / margin ?

- évite les recalculs à la volée → meilleure performance;
- garantit qu'analystes et rapports parlent du *même* CA / de la *même* marge.

Degenerate dimensions. `commande_id` et `numero_ligne` sont conservés dans `fait_ventes` sans table dimensionnelle associée — ce sont des **degenerate dimensions** : utiles pour drill-down jusqu'à la commande, mais sans attribut propre.

3.5 Bus matrix

Processus métier	date	produit	client	employe	fournisseur	canal	geographie
Vente (ligne de commande)	✓	✓	✓	✓	✓	✓	✓ (client)

Table 4 – Bus matrix actuel : un seul processus. L'ajout futur de *Réception fournisseur* ou *Mouvement de stock* partagerait `dim_produit`, `dim_fournisseur`, `dim_date`, `dim_geographie`.

3.6 Vue synthétique de la transition

1	OLTP (3NF à ops)	ETL (T-SQL à OrionETL)	DWH (étoile à dw)
2			
3	ops.ligne_produit		
4	ops.categorie_produit aplatit dim.dim_produit (SCD1)		dw.dim_produit
5	ops.groupe_produit (full reload)		
6	ops.produit		
7	ops.fournisseur	dim.dim_fournisseur (SCD1)	dw.dim_fournisseur
8	ops.client + groupe + ville + ...	dim.dim_client (SCD2)	dw.dim_client
9	ops.employe + org_* + manager	dim.dim_employe (SCD2)	dw.dim_employe
10	ops.continent/pays/region/ville	dim.dim_geographie (SCD1)	dw.dim_geographie
11	ops.canal_vente	dim.dim_canal (SCD1)	dw.dim_canal
12	(série de dates 1997-2003)	dim.dim_date (statique)	dw.dim_date
13	ops.commande + ligne_commande	fact.fait_ventes (incr. + mesures)	dw.fait_ventes
14		etl.watermark, etl.run_log	(méta)

4 Modélisation dimensionnelle (DWH)

4.1 Étoile vs flocon

Pour cet entrepôt, le **schéma en étoile** a été préféré au flocon pour trois raisons :

1. **Performance** de lecture : moins de jointures sur les requêtes OLAP, qui dominent la charge.
2. **Lisibilité** pour les analystes : une dimension = une seule table avec tous les attributs hiérarchiques aplatis.
3. **Simplicité d'ETL** : les hiérarchies ne sont reconstruites qu'une fois, à l'extraction.

La sur-redondance induite est négligeable face aux gains de simplicité.

4.2 Schéma en étoile

4.3 Stratégie SCD (Slowly Changing Dimensions)

Toutes les dimensions ne sont pas traitées de la même manière : certaines peuvent être écrasées (SCD1), d'autres doivent conserver l'historique des versions (SCD2).

Dimension	SCD	Justification
dim_date	–	Statique, peuplée une seule fois (1997–2003).
dim_produit	1	Le nom et la hiérarchie évoluent peu ; on écrase.
dim_client	2	Le groupe d'achat et l'adresse évoluent ; on veut analyser les ventes <i>telles qu'à l'époque</i> .
dim_employe	2	Salaire, section, manager évoluent ; la RH demande la traçabilité.
dim_fournisseur	1	Très peu de changements en pratique.
dim_canal	1	Référentiel quasi-statique.
dim_geographie	1	Référentiel.

Table 5 – Choix SCD par dimension.

Implémentation SCD2. Chaque dimension SCD2 porte les colonnes : `effectif_du`, `effectif_au`, `est_courant`, `hash_ligne` (sha256 stable des attributs SCD). Le job ETL compare le hash de la ligne source au hash de la version courante en base. S'ils diffèrent, l'ancienne version est close (`est_courant` = FALSE, `effectif_au` = `ajd`) et une nouvelle version est insérée.

4.4 Mesures de la fact

Les mesures sont pré-calculées au chargement pour éviter des opérations arithmétiques répétées à la lecture :

$$\begin{aligned}\text{montant_brut} &= \text{quantite} \times \text{prix_unitaire} \\ \text{montant_remise} &= \text{montant_brut} \times \text{pct_remise} \\ \text{montant_net} &= \text{montant_brut} - \text{montant_remise} \\ \text{montant_cout} &= \text{quantite} \times \text{cout_unitaire} \\ \text{montant_marge} &= \text{montant_net} - \text{montant_cout}\end{aligned}$$

`montant_net` est la convention retenue pour le *chiffre d'affaires* et `montant_marge` pour la *marge*. Les mesures additives (`quantite`, `montant_*`) peuvent être agrégées librement ; les non-additives (`prix_unitaire`, `cout_unitaire`, `pct_remise`) ne sont utilisables qu'au grain ligne ou via une moyenne pondérée.

5 Architecture technique : stack Docker

5.1 Vue d'ensemble

L'environnement complet est provisionné par un unique *docker-compose.yml* qui démarre **six services** sur un réseau interne *orion-net* :

- *postgres-oltp* : la base opérationnelle *Orion* (Postgres);
- *postgres-dwh* : l'entrepôt de données en étoile (Postgres);
- *mssql-etl* : SQL Server 2022, qui héberge la base *OrionETL* et toutes les procédures stockées T-SQL de transformation;
- *orchestrator* : conteneur Python long-running, transporte les données entre Postgres et SQL Server;
- *data-gen* : conteneur Python one-shot peuplant l'OLTP via Faker (-profile seed);
- *pgadmin* : UI web pour explorer les deux bases Postgres.

5.2 docker-compose.yml (extrait)

Listing 3 – Services principaux (docker-compose.yml)

```
1 services:
2   postgres-oltp:
3     image: postgres:16-alpine
4     environment:
5       POSTGRES_DB: ${OLTP_DB}
6       POSTGRES_USER: ${OLTP_USER}
7       POSTGRES_PASSWORD: ${OLTP_PASSWORD}
8     ports: [ "${OLTP_PORT}:5432" ]
9     volumes:
10      - oltp_data:/var/lib/postgresql/data
11      - ./oltp/init:/docker-entrypoint-initdb.d:ro
12     healthcheck:
13       test: [ "CMD-SHELL", "pg_isready -U ${OLTP_USER} -d ${OLTP_DB}" ]
14
15   postgres-dwh:
16     image: postgres:16-alpine
17     environment: { POSTGRES_DB: ${DWH_DB}, ... }
18     volumes:
19      - dwh_data:/var/lib/postgresql/data
20      - ./dwh/init:/docker-entrypoint-initdb.d:ro
21
22   mssql-etl:                                     # moteur ETL : SQL Server + procs T-SQL
23     build: ./mssql-etl
24     environment:
25       ACCEPT_EULA: "Y"
26       MSSQL_PID: ${MSSQL_PID}
27       MSSQL_SA_PASSWORD: ${MSSQL_SA_PASSWORD}
28     ports: [ "${MSSQL_PORT}:1433" ]
29     volumes: [ mssql_data:/var/opt/mssql ]
30
31   orchestrator:                                 # transporte Postgres <-> SQL Server
32     build: ./orchestrator
33     depends_on:
34       postgres-oltp: { condition: service_healthy }
35       postgres-dwh: { condition: service_healthy }
36       mssql-etl: { condition: service_healthy }
37     restart: unless-stopped
38
39   data-gen:
40     build: ./data-gen
41     profiles: [ "seed" ]                         # ne tourne que sur demande explicite
```

Healthchecks. L'orchestrator ne démarre que lorsque les trois bases passent leur healthcheck (pg_isready pour Postgres, SELECT 1 via sqlcmd pour SQL Server). Le service data-gen utilise un profile pour ne pas se lancer automatiquement (docker compose -profile seed run -rm data-gen).

Volumes nommés. oltp_data, dwh_data, mssql_data, pgadmin_data : la persistance survit à docker compose down mais peut être effacée par down -v.

5.3 Configuration et démarrage

Toute la configuration est centralisée dans un fichier `.env` versionné qui contient les volumétries, les ports, les credentials SQL Server et les paramètres cron de l'ETL.

Listing 4 – Démarrage de la stack

```
1 # 1) Bases + SQL Server + orchestrator + pgAdmin
2 docker compose up -d --build
3
4 # 2) Peuplement OLTP (one-shot)
5 docker compose --profile seed run --rm data-gen
6
7 # 3) Forcer un cycle ETL immediat (sinon il s'execute selon le cron)
8 docker compose restart orchestrator
9
10 # 4) Voir les logs ETL (orchestrateur Python + SQL Server)
11 docker compose logs -f orchestrator
12 docker compose logs -f mssql-etl
```

6 Génération des données opérationnelles

6.1 Stratégie

L'énoncé évoque 5 500 produits, 90 000 clients et 980 000 commandes. Pour rester dans des temps de cycle raisonnables en TP, le générateur produit des volumes *paramétrés* via le `.env` :

Entité	Cible énoncé	Défaut <code>.env</code>
employes	≈700	700
fournisseurs	64	64
produits	≈5 500	5 500
clients	≈90 000	90 000
commandes	≈980 000	980 000
lignes_commande	≈2,5M	~2,5M

Table 6 – Volumétries du générateur par défaut, alignées sur l'énoncé. Pour un cycle de TP plus rapide, ramener `GEN_NB_ORDERS` à 50 000 dans `.env`.

6.2 Réalisme injecté

- Hiérarchie produit : 5 lignes × 12 catégories × 24 groupes, *insérée par les seeds statiques*, puis affectation aléatoire des produits aux groupes.
- Historique de prix : 1 à 3 fenêtres temporelles par produit, marge cible entre 30 % et 150 % sur le coût.
- 30 % des produits ont au moins une remise (5 à 30 % sur 7 à 60 jours).
- 25 % des clients possèdent une carte *Orion Star Club*.
- Saisonnalité : surcharge de 30 % sur les commandes des mois de novembre et décembre.
- Les commandes sont passées prioritairement par des employés du département « Sales ».
- `total_amount` est recalculé en SQL après l'insertion des lignes pour rester cohérent avec les remises appliquées.

6.3 Listing : extrait du générateur

Listing 5 – Génération des prix historisés

```
1 for pid in produit_ids:
2     windows = []
3     cursor_d = DATE_DEBUT - timedelta(days=180)
4     while cursor_d < DATE_FIN:
5         length = timedelta(days=random.randint(180, 720))
6         end_d = min(cursor_d + length, DATE_FIN + timedelta(days=365))
7         windows.append((cursor_d, end_d))
8         cursor_d = end_d + timedelta(days=1)
9     for i, (sd, ed) in enumerate(windows):
10        cout = round(random.uniform(5, 200), 2)
11        prix = round(cout * random.uniform(1.3, 2.5), 2)
12        # La derniere fenetre reste ouverte (date_fin IS NULL)
13        end_val = None if i == len(windows) - 1 else ed
14        prix_rows.append((pid, sd, end_val, cout, prix))
```

7 ETL T-SQL sur SQL Server

7.1 Architecture en deux niveaux

L'ETL Orion est volontairement scindé en deux niveaux complémentaires :

Couche	Rôle	Techno
Logique de transformation	TRUNCATE/INSERT/MERGE, calcul SCD2, calcul des mesures, journal	T-SQL (procédures stockées sur SQL Server)
Transport / orchestration	Extract Postgres OLTP, push staging SQL Server, EXEC procs, pull SQL Server, COPY DWH	Python (orchestrate.py + APScheduler)

Table 7 – Séparation logique métier (T-SQL) vs transport (Python).

Toute la **logique métier ETL** est concentrée dans des procédures stockées T-SQL — elles sont versionnées, testables individuellement (EXEC etl.sp_load_dim_client), et journalisées dans etl.run_log. L'orchestrateur Python est un **transporteur** neutre, sans logique de transformation.

7.2 Procédures stockées T-SQL

Procédure	Mode	Cadence
etl.sp_load_dim_date	bootstrap (idempotent)	une fois
etl.sp_load_dim_canal	full reload	quotidien
etl.sp_load_dim_geographie	full reload	quotidien
etl.sp_load_dim_fournisseur	full reload	quotidien
etl.sp_load_dim_produit	full reload (SCD1)	quotidien
etl.sp_load_dim_client	SCD2 merge (hash_ligne)	quotidien
etl.sp_load_dim_employe	SCD2 merge (hash_ligne)	quotidien
etl.sp_load_fait_ventes	incrémental (watermark)	quotidien
etl.sp_run_pipeline	enchaîne tout	quotidien

Table 8 – Procédures stockées T-SQL et fréquence d'exécution.

7.3 Cur SCD2 en T-SQL

Listing 6 – Mise à jour SCD2 dans sp_load_dim_client (extrait)

```
1 ;WITH src AS (  
2     SELECT client_id, nom_complet, sexe, tranche_age, groupe_client, ...,  
3           CONVERT(CHAR(64), HASHBYTES('SHA2_256',  
4             CONCAT_WS(N'|', nom_complet, sexe, tranche_age, groupe_client, ...)  
5           ), 2) AS hash_ligne  
6     FROM   staging.client_full  
7 )  
8 -- 1) Fermer la version courante quand le hash change  
9 UPDATE d  
10    SET effectif_au = @ajd, est_courant = 0  
11    FROM dim.dim_client d  
12   JOIN src      s ON s.client_id = d.client_id  
13  WHERE d.est_courant = 1 AND d.hash_ligne <> s.hash_ligne;  
14  
15 -- 2) Insérer la nouvelle version (ou la première version)  
16 INSERT dim.dim_client (... , effectif_du, effectif_au, est_courant, hash_ligne)  
17 SELECT s.client_id, ..., @ajd, NULL, 1, s.hash_ligne  
18    FROM src s  
19   LEFT JOIN dim.dim_client d
```

```

20     ON d.client_id = s.client_id AND d.est_courant = 1
21 WHERE d.cle_client IS NULL OR d.hash_ligne <> s.hash_ligne;

```

7.4 Watermark et idempotence

Principe. La table `etl.watermark(job_name, last_value)` côté SQL Server mémorise pour chaque job la *borne haute* de la dernière extraction. Le job `sp_load_fact_sales` consomme uniquement les lignes `order_date > watermark`, puis met à jour le watermark via `MERGE` à `MAX(order_date)` ingéré. Cela garantit :

- **Pas de doublon** : la procédure commence par un `DELETE` ciblé sur `(order_id, line_no)` en cas de re-jeu accidentel d'un même lot.
- **Pas de perte** : c'est `order_date` qui est tracké car il représente la dimension métier ; les commandes « back-datées » sont rares dans Orion (énoncé).
- **Reprise sur erreur** : tout est dans une transaction T-SQL ; en cas d'échec, le watermark n'est pas avancé.

7.5 Orchestrateur Python

L'orchestrateur (`orchestrator/orchestrate.py`, ~200 lignes) fait trois choses : *extract*, *exec*, *push*. Il est planifié par **APScheduler** (cron-like in-process) ; pas d'Airflow, pas de Cron système.

Listing 7 – Cycle ETL côté orchestrateur

```

1 def run_full_pipeline():
2     with both_connections() as (pg_oltp, pg_dwh, ms):
3         load_staging(pg_oltp, ms)    # 1) extract OLTP -> staging SQL Server
4         run_pipeline(ms)             # 2) EXEC etl.sp_run_pipeline (T-SQL)
5         push_to_dwh(ms, pg_dwh)     # 3) export dim/fact -> Postgres DWH
6
7 sched = BlockingScheduler(timezone="UTC")
8 sched.add_job(run_full_pipeline,
9               CronTrigger(hour=ETL_HOUR, minute=ETL_MINUTE),
10              id="etl_daily")

```

7.6 Journalisation

Chaque procédure ouvre une ligne dans `etl.run_log` via `etl.sp_run_start` et la clôt via `etl.sp_run_end` avec les champs `rows_in`, `rows_out`, `error_msg` :

Listing 8 – Journal d'exécution côté SQL Server

```

1 SELECT TOP 20 job_name, started_at, ended_at, status,
2             rows_in, rows_out, LEFT(error_msg,80) AS error_msg
3 FROM OrionETL.etl.run_log
4 ORDER BY run_id DESC;

```

7.7 Variante « tout SQL Server »

L'image `mssql-etl` embarque déjà `unixODBC` et le driver `psqlodbc` : on peut donc créer un **Linked Server** PostgreSQL et faire des `OPENQUERY('ORION_OLTP_LINK', '...')` directement depuis T-SQL, supprimant l'orchestrateur Python. Cette variante n'a pas été retenue pour le TP (configuration ODBC plus fragile, moins de visibilité dans les logs Docker), mais le rapport documente l'option.

8 Réponses aux questions analytiques

Les requêtes ci-dessous tournent sur `orion_dwh` (schéma `dw`). Elles sont également regroupées dans `analytics/queries.sql` pour exécution en lot.

Q1. Quels sont les produits qui se vendent le mieux ?

Top 20 par quantité vendue, toutes périodes confondues.

```
1 SELECT p.product_id, p.product_name, p.product_group_name,
2        SUM(f.quantity) AS qty_total,
3        SUM(f.net_amount) AS revenue
4 FROM   fact_sales f JOIN dim_product p USING (product_key)
5 GROUP BY p.product_id, p.product_name, p.product_group_name
6 ORDER BY qty_total DESC
7 LIMIT 20;
```

Lecture. `qty_total` mesure le volume, `revenue` la valeur. On peut classer alternativement par `revenue` pour répondre « en valeur » plutôt qu'en « en volume ».

Q2. Quels sont les produits en perte de vitesse ?

Variation négative de CA d'une année sur l'autre, pire descente en tête.

```
1 WITH yearly AS (
2   SELECT p.product_id, p.product_name, d.year_num,
3          SUM(f.net_amount) AS revenue
4   FROM   fact_sales f
5   JOIN   dim_product p USING (product_key)
6   JOIN   dim_date    d USING (date_key)
7   GROUP BY p.product_id, p.product_name, d.year_num
8 ), delta AS (
9   SELECT product_id, product_name, year_num, revenue,
10          LAG(revenue) OVER (PARTITION BY product_id
11                             ORDER BY year_num) AS revenue_prev
12 FROM   yearly
13 )
14 SELECT product_id, product_name, year_num,
15        revenue, revenue_prev,
16        revenue - revenue_prev AS delta_abs,
17        CASE WHEN revenue_prev > 0
18              THEN ROUND(100*(revenue - revenue_prev)/revenue_prev, 2)
19              END AS delta_pct
20 FROM   delta
21 WHERE  revenue_prev IS NOT NULL AND revenue < revenue_prev
22 ORDER BY delta_pct ASC NULLS LAST
23 LIMIT 20;
```

Q3. Produits qui contribuent très peu au CA pour un pays / une année donnés ?

Critère : moins de 0,1 % du CA pays-année.

```
1 WITH base AS (
2   SELECT p.product_id, p.product_name, SUM(f.net_amount) AS revenue
3   FROM   fact_sales f
4   JOIN   dim_product p USING (product_key)
5   JOIN   dim_geography g ON g.geography_key = f.geography_cust_key
6   JOIN   dim_date d USING (date_key)
7   WHERE  g.country_name = 'France' -- :pays
8          AND d.year_num = 2002 -- :annee
9   GROUP BY p.product_id, p.product_name
10 ), total AS (SELECT SUM(revenue) AS tot FROM base)
11 SELECT b.product_id, b.product_name, b.revenue,
```



```

12         ROUND(100*b.revenue/NULLIF(t.tot,0), 4) AS pct_ca
13 FROM   base b CROSS JOIN total t
14 WHERE  b.revenue/NULLIF(t.tot,0) < 0.001
15 ORDER BY pct_ca ASC;

```

Décision commerciale. Cette même requête sert à proposer une remise saisonnière sur les produits identifiés (insertion automatique dans ops.product_discount via un job dédié, hors périmètre du TP).

Q4. Marge générée par un groupe de produits, par année

```

1 SELECT  p.product_group_name, d.year_num,
2         SUM(f.net_amount) AS revenue,
3         SUM(f.cost_amount) AS cogs,
4         SUM(f.margin_amount) AS margin,
5         ROUND(100*SUM(f.margin_amount)/NULLIF(SUM(f.net_amount),0), 2) AS margin_pct
6 FROM    fact_sales f
7 JOIN    dim_product p USING (product_key)
8 JOIN    dim_date d USING (date_key)
9 WHERE   p.product_group_name = 'Premium Hiking' -- :groupe
10 GROUP BY p.product_group_name, d.year_num
11 ORDER BY d.year_num;

```

Q5. La marge dépend-elle de la quantité vendue ?

Corrélation de Pearson + régression linéaire avec les fonctions Postgres natives.

```

1 WITH per_product AS (
2     SELECT p.product_id,
3           SUM(f.quantity) AS qty,
4           SUM(f.margin_amount) AS margin
5 FROM     fact_sales f JOIN dim_product p USING (product_key)
6 GROUP BY p.product_id
7 )
8 SELECT  CORR(qty, margin) AS pearson_qty_margin,
9         REGR_SLOPE(margin, qty) AS slope,
10        REGR_INTERCEPT(margin, qty) AS intercept,
11        REGR_R2(margin, qty) AS r_squared
12 FROM    per_product;

```

Q6. Les remises font-elles augmenter les ventes (Q6) et la marge (Q7) ?

Comparaison directe des lignes avec et sans remise.

```

1 SELECT  CASE WHEN discount_pct > 0 THEN 'avec_remise' ELSE 'sans_remise' END AS bucket,
2         COUNT(*) AS nb_lignes,
3         ROUND(AVG(quantity)::numeric, 3) AS qty_moy_ligne,
4         ROUND(AVG(net_amount)::numeric, 2) AS ca_moy_ligne,
5         ROUND(AVG(margin_amount)::numeric,2) AS marge_moy_ligne,
6         ROUND(SUM(net_amount)::numeric, 2) AS ca_total,
7         ROUND(SUM(margin_amount)::numeric,2) AS marge_totale
8 FROM    fact_sales
9 GROUP BY 1;

```

Limite. Cette comparaison est descriptive, non causale : pour établir un effet causal il faudrait un dispositif d'A/B testing ou un *difference-in-differences* sur des produits pilotes.

Q7. Commerciaux qui font le plus de ventes

```

1 SELECT  e.employee_id, e.full_name, e.org_country,
2         COUNT(DISTINCT f.order_id) AS nb_commandes,

```

```

3      SUM(f.net_amount)          AS ca,
4      SUM(f.margin_amount)      AS marge
5 FROM    fact_sales f
6 JOIN    dim_employee e ON e.employee_key = f.employee_key
7 WHERE   e.is_current
8 GROUP   BY 1, 2, 3
9 ORDER   BY ca DESC
10 LIMIT  20;

```

Q8. Commerciaux les plus performants par pays / sexe / âge / salaire

```

1 WITH base AS (
2     SELECT e.employee_id, e.full_name, e.org_country,
3           e.gender, e.age_band, e.salary_band,
4           SUM(f.net_amount) AS ca
5 FROM    fact_sales f JOIN dim_employee e ON e.employee_key = f.employee_key
6 WHERE   e.is_current
7 GROUP   BY 1, 2, 3, 4, 5, 6
8 )
9 SELECT org_country, gender, age_band, salary_band,
10        employee_id, full_name, ca,
11        ROW_NUMBER() OVER (PARTITION BY org_country ORDER BY ca DESC) AS rang_pays,
12        ROW_NUMBER() OVER (PARTITION BY gender ORDER BY ca DESC) AS rang_sexe,
13        ROW_NUMBER() OVER (PARTITION BY age_band ORDER BY ca DESC) AS rang_age,
14        ROW_NUMBER() OVER (PARTITION BY salary_band ORDER BY ca DESC) AS rang_salaire
15 FROM    base
16 ORDER   BY ca DESC;

```

Q9. Quels groupes de clients sont identifiés ?

```

1 SELECT c.customer_group,
2        COUNT(DISTINCT c.customer_id) AS nb_clients,
3        SUM(f.net_amount) AS ca,
4        ROUND(AVG(f.net_amount)::numeric, 2) AS panier_moyen_ligne
5 FROM    fact_sales f JOIN dim_customer c ON c.customer_key = f.customer_key
6 WHERE   c.is_current
7 GROUP   BY c.customer_group
8 ORDER   BY ca DESC;

```

Note. Les groupes de référence sont *Bronze*, *Silver*, *Gold*, *VIP*, *Inactif*, peuplés dans le fichier de seeds statiques de l'OLTP.

Q10. Clients les plus rentables (top 50)

```

1 SELECT c.customer_id, c.full_name, c.country_name,
2        SUM(f.net_amount) AS ca,
3        SUM(f.margin_amount) AS marge,
4        COUNT(DISTINCT f.order_id) AS nb_commandes
5 FROM    fact_sales f JOIN dim_customer c ON c.customer_key = f.customer_key
6 WHERE   c.is_current
7 GROUP   BY 1, 2, 3
8 ORDER   BY marge DESC
9 LIMIT  50;

```

Q11. Fournisseurs proposant des produits rentables

```

1 SELECT s.supplier_id, s.supplier_name, s.country_name,
2        SUM(f.net_amount) AS ca,
3        SUM(f.margin_amount) AS marge,
4        ROUND(100*SUM(f.margin_amount)/NULLIF(SUM(f.net_amount),0),2) AS taux_marge
5 FROM    fact_sales f JOIN dim_supplier s USING (supplier_key)

```

```

6 GROUP BY 1, 2, 3
7 ORDER BY marge DESC
8 LIMIT 20;

```

Q12. Moyenne et écart-type du chiffre d'affaires

Trois grains : par commande, par mois, par employé.

```

1 -- Par commande
2 WITH per_order AS (
3     SELECT order_id, SUM(net_amount) AS ca FROM fact_sales GROUP BY order_id
4 )
5 SELECT AVG(ca) AS ca_moy, STDDEV_SAMP(ca) AS ca_std,
6         MIN(ca) AS ca_min, MAX(ca) AS ca_max
7 FROM   per_order;
8
9 -- Par mois
10 WITH per_month AS (
11     SELECT d.year_num, d.month_num, SUM(f.net_amount) AS ca
12     FROM   fact_sales f JOIN dim_date d USING (date_key)
13     GROUP BY d.year_num, d.month_num
14 )
15 SELECT AVG(ca) AS ca_moy_mois, STDDEV_SAMP(ca) AS ca_std_mois FROM per_month;
16
17 -- Par employé
18 WITH per_emp AS (
19     SELECT e.employee_id, SUM(f.net_amount) AS ca
20     FROM   fact_sales f JOIN dim_employee e ON e.employee_key = f.employee_key
21     WHERE  e.is_current
22     GROUP BY e.employee_id
23 )
24 SELECT AVG(ca) AS ca_moy_emp, STDDEV_SAMP(ca) AS ca_std_emp FROM per_emp;

```

Q13. Variables qui expliquent le mieux le chiffre d'affaires

Calcul du η^2 (eta-squared) = part de variance inter-groupes. Plus η^2 est proche de 1, plus la variable est explicative.

$$\eta^2 = \frac{\sum_g n_g (\bar{y}_g - \bar{y})^2}{\sum_i (y_i - \bar{y})^2}$$

```

1 WITH facts AS (
2     SELECT f.net_amount,
3            p.product_line_name, p.product_category_name,
4            c.customer_group, ch.channel_code,
5            e.gender AS emp_gender, e.salary_band AS emp_salary_band,
6            g.country_name AS cust_country
7 FROM   fact_sales f
8 JOIN   dim_product p USING (product_key)
9 JOIN   dim_customer c ON c.customer_key = f.customer_key
10 JOIN  dim_channel ch USING (channel_key)
11 JOIN  dim_employee e ON e.employee_key = f.employee_key
12 JOIN  dim_geography g ON g.geography_key = f.geography_cust_key
13 ), mu_t AS (SELECT AVG(net_amount) AS m_t FROM facts),
14 sst AS (SELECT SUM(power(net_amount-m_t,2)) AS st FROM facts, mu_t),
15 eta(variable, eta_squared) AS (
16     SELECT 'product_line', (
17         SELECT SUM(n_g*power(m_g-m_t,2))/NULLIF(st,0)
18         FROM (SELECT product_line_name, AVG(net_amount) m_g, COUNT(*) n_g
19              FROM facts GROUP BY product_line_name) g, mu_t, sst)
20     UNION ALL SELECT 'customer_group', (...) -- etc.
21 )
22 SELECT variable, ROUND(eta_squared::numeric, 4) AS eta_squared

```

```
23 FROM eta ORDER BY eta_squared DESC NULLS LAST;
```

Lecture. On obtient un classement des variables candidates par pouvoir explicatif. Pour aller plus loin (régression linéaire ou non-linéaire), on exporte vers Python/R via Pandas.

Q14. Différence significative de CA entre commerciaux femmes et hommes ?

Statistique de Welch (test t pour variances inégales).

```
1 WITH per_emp AS (  
2   SELECT e.gender, e.employee_id, SUM(f.net_amount) AS ca  
3   FROM   fact_sales f JOIN dim_employee e ON e.employee_key = f.employee_key  
4   WHERE  e.is_current  
5   GROUP BY e.gender, e.employee_id  
6 ), agg AS (  
7   SELECT gender, COUNT(*) AS n, AVG(ca) AS mean_ca, VAR_SAMP(ca) AS var_ca  
8   FROM   per_emp GROUP BY gender  
9 )  
10 SELECT f.mean_ca AS mean_F, m.mean_ca AS mean_M,  
11        f.n       AS n_F,    m.n       AS n_M,  
12        (f.mean_ca - m.mean_ca)  
13        / sqrt(f.var_ca/f.n + m.var_ca/m.n) AS welch_t  
14 FROM   (SELECT * FROM agg WHERE gender='F') f  
15 CROSS JOIN (SELECT * FROM agg WHERE gender='M') m;
```

Décision. La p -value associée à t se calcule avec la loi de Student aux degrés de liberté de Welch ; comme PostgreSQL n'embarque pas `pt()`, on le fait en SciPy après export, ou on installe l'extension `tablefunc` / `pgsl-stats`.

Annexe A — Arborescence du projet

```
1 tp1-orion/
2 |-- README.md
3 |-- docker-compose.yml
4 |-- .env
5 |-- doc/
6 |   |-- MODELISATION.md           (modele OP + transition + DW)
7 |   |-- UML.md                   (11 diagrammes UML)
8 |   |-- pgadmin-servers.json      (auto-config pgAdmin)
9 |   |-- rapport/
10 |     |-- rapport.tex             (rapport principal A4)
11 |     |-- rapport.pdf
12 |     |-- uml-poster.tex          (poster A0 : 11 diag. UML)
13 |     |-- uml-individual.tex      (1 page A3 par diagramme)
14 |     |-- diagrams/              (sources Mermaid .mmd)
15 |     |-- screenshots/           (PNG des captures d'écran)
16 |     |-- build/                 (PDF compiles)
17 |-- oltp/
18 |   |-- init/
19 |     |-- 01_schema.sql           (DDL OLTP, schema "ops")
20 |     |-- 02_static_data.sql      (referentiels statiques)
21 |-- dwh/
22 |   |-- init/
23 |     |-- 01_schema.sql           (DDL DWH, schema "dw")
24 |-- data-gen/
25 |   |-- Dockerfile
26 |   |-- requirements.txt
27 |   |-- generate.py               (Faker, peuplement OLTP)
28 |-- mssql-etl/                   ** moteur ETL : SQL Server 2022 **
29 |   |-- Dockerfile               (mssql/server + unixODBC + psqlodbc)
30 |   |-- scripts/
31 |     |-- entrypoint.sh           (start sqlservr, run init/*.sql)
32 |     |-- init-db.sh             (re-execution manuelle)
33 |   |-- init/                    (7 scripts T-SQL, executes en ordre)
34 |     |-- 01_database.sql         (CREATE DATABASE OrionETL + schemas)
35 |     |-- 02_meta_tables.sql      (etl.run_log, etl.watermark, helpers)
36 |     |-- 03_staging.sql          (staging.* miroir denormalise OLTP)
37 |     |-- 04_dim_tables.sql       (dim.* + fact.* gold)
38 |     |-- 05_sp_dim_simple.sql    (sp_load_dim_date/channel/geo/...)
39 |     |-- 06_sp_dim_scd2.sql      (sp_load_dim_customer/employee SCD2)
40 |     |-- 07_sp_fact.sql          (sp_load_fact_sales + sp_run_pipeline)
41 |-- orchestrator/                ** transport Postgres <-> SQL Server **
42 |   |-- Dockerfile               (python:3.12 + msodbcsql18)
43 |   |-- requirements.txt
44 |   |-- orchestrate.py            (extract / EXEC / push, APScheduler)
45 |-- analytics/
46 |   |-- queries.sql               (15 requetes repondant aux questions)
```

Annexe B — Démarrage et exploitation

Listing 9 – Cycle de vie complet

```
1 # Demarrer (build des images mssql-etl + orchestrator inclus)
2 cd ~/dev-laboratory/tp-ed-eneam/tp1-orion
3 docker compose up -d --build
4
5 # Peupler l'OLTP avec Faker (one-shot)
6 docker compose --profile seed run --rm data-gen
7
8 # Forcer un cycle ETL immediat
9 docker compose restart orchestrator
10
11 # Inspecter
12 docker compose ps
13 docker compose logs -f orchestrator
14 docker compose logs -f mssql-etl
15
16 # Journal SQL Server
17 docker compose exec mssql-etl /opt/mssql-tools18/bin/sqlcmd \
```

```
18 -S localhost -U sa -P "$MSSQL_SA_PASSWORD" -No -d OrionETL \  
19 -Q "SELECT TOP 10 job_name, started_at, status, rows_in, rows_out  
20 FROM etl.run_log ORDER BY run_id DESC"  
21  
22 # Resultat dans le DWH  
23 docker compose exec postgres-dwh psql -U dwh -d orion_dwh \  
24 -c "SELECT COUNT(*) FROM dw.fait_ventes;"  
25  
26 # Reset complet (efface les volumes)  
27 docker compose down -v
```

Fin du rapport.

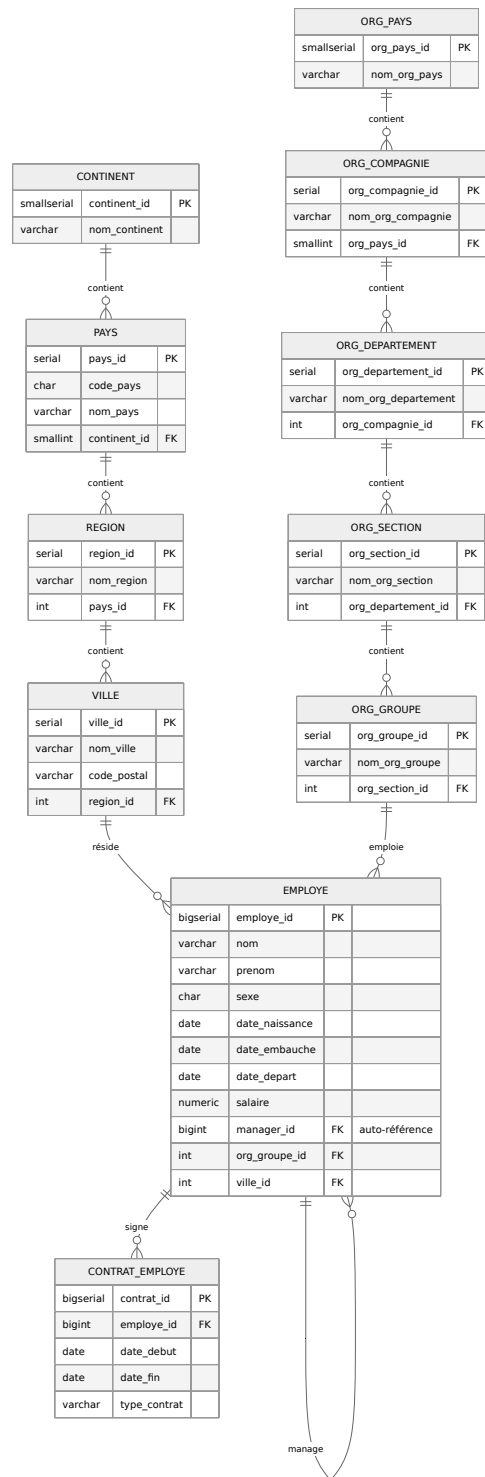


Figure 2 – Sous-domaine *Géographie & Organisation*. La table **employee** hérite de la position dans les deux hiérarchies via **org_group_id** et **city_id**, et porte une auto-référence **manager_id**.

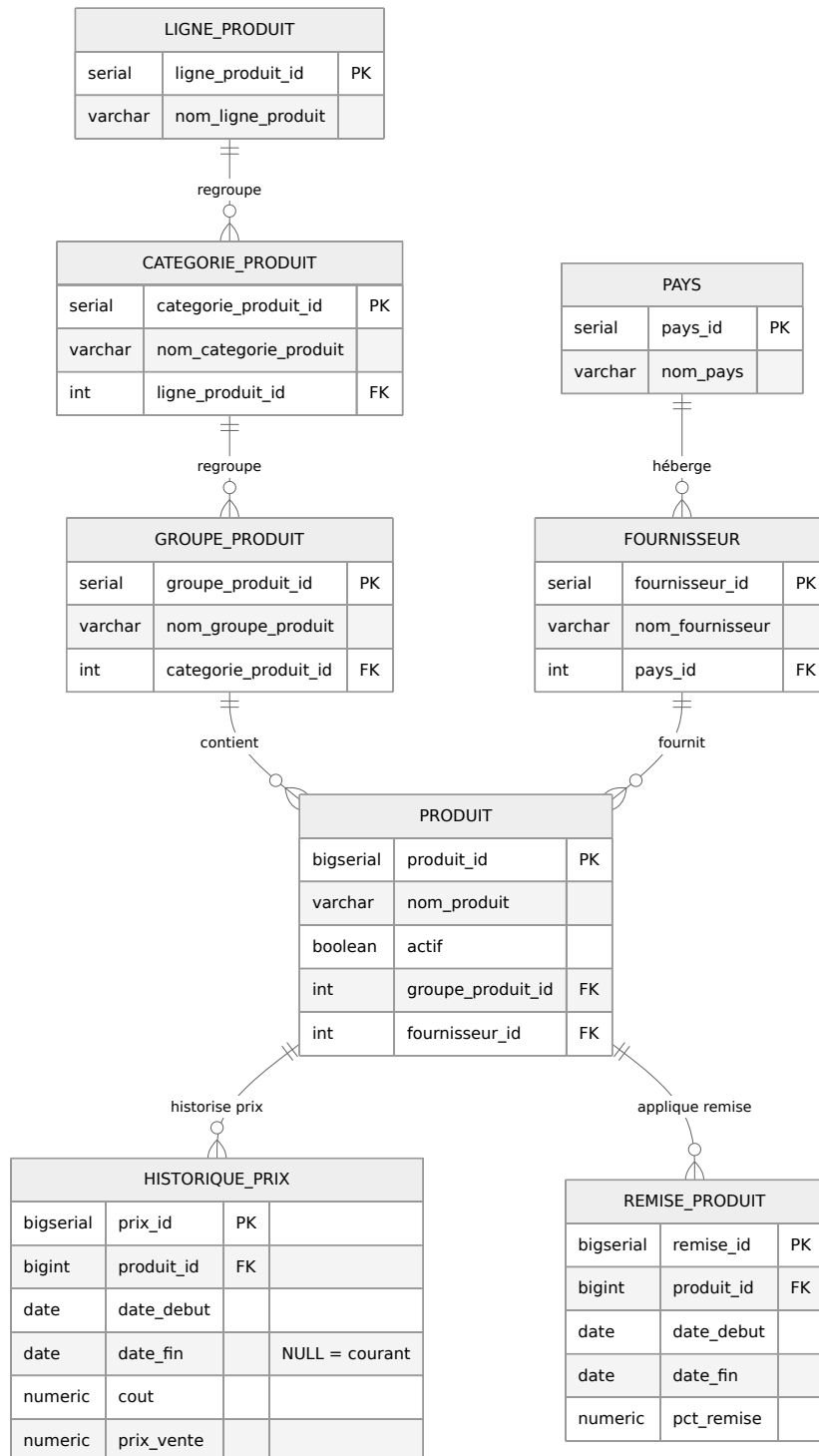


Figure 3 – Sous-domaine *Produits & Fournisseurs*. La hiérarchie comporte 4 niveaux; `product_price_history` et `product_discount` capturent les variations temporelles (prix et remises).

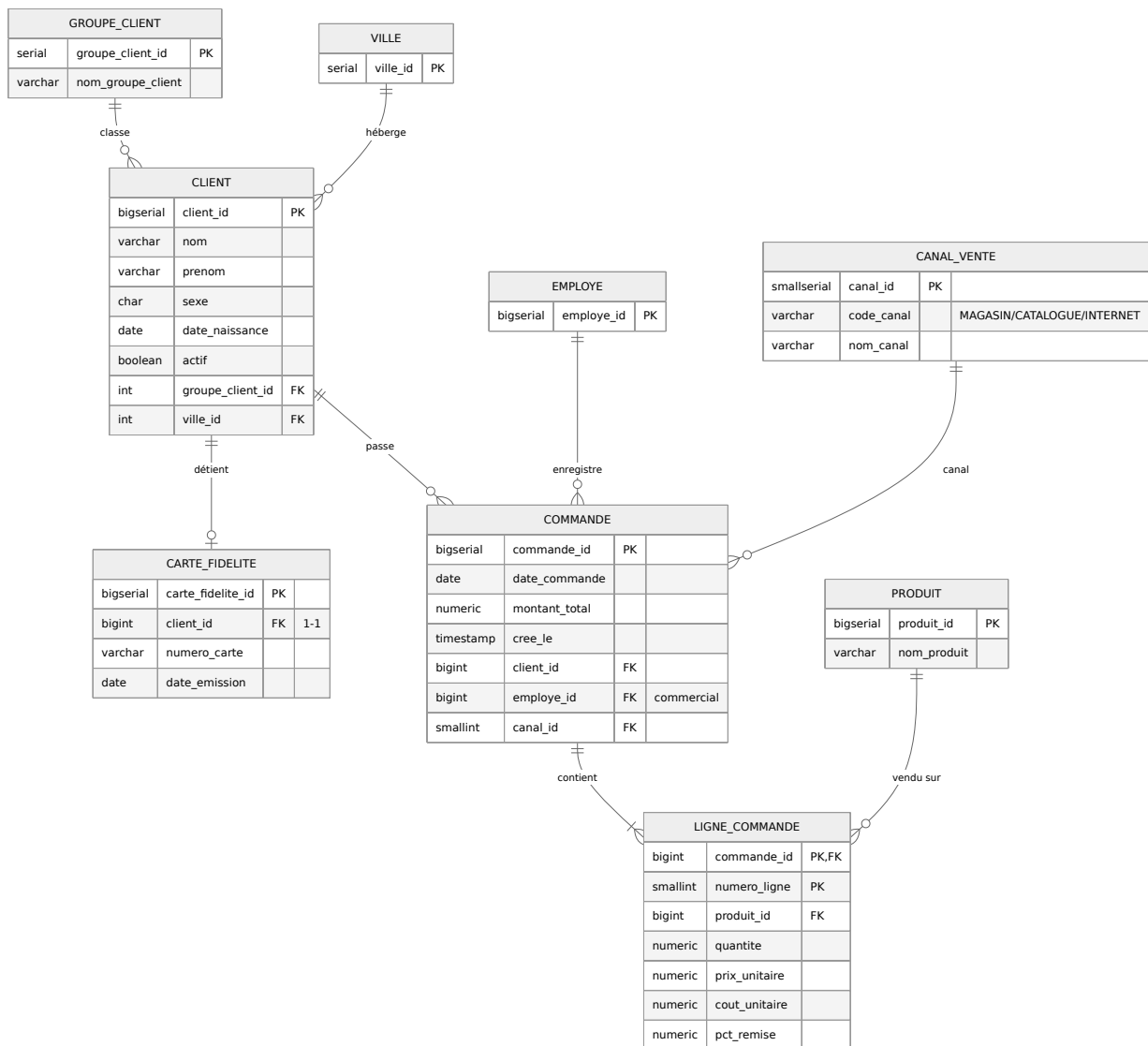


Figure 4 – Sous-domaine *Clients & Commandes*. La table des lignes a une clé primaire composite (*order_id*, *line_no*) ; chaque ligne pointe vers un produit, et l’en-tête de commande vers un client, un commercial et un canal.

• **Capture d’écran à insérer**

screenshots/transition-pgadmin-vs-mssms.png

Vue côte à côte : le schéma ops normalisé sur pgAdmin (à gauche) et la base OrionETL (staging, dim, fact) sur SSMS (à droite).

Figure 5 – Vue côte à côte : le schéma ops normalisé sur pgAdmin (à gauche) et la base OrionETL (staging, dim, fact) sur SSMS (à droite).

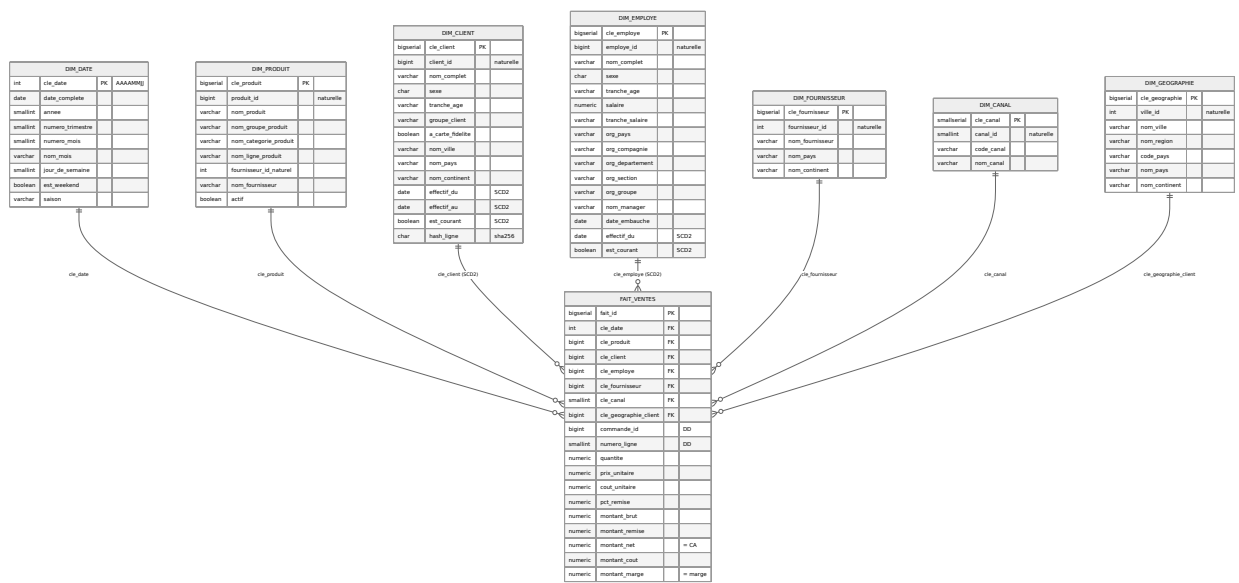


Figure 6 – Schéma en étoile centré sur **fact_sales** (grain = ligne de commande). Les dimensions SCD2 sont identifiées explicitement.

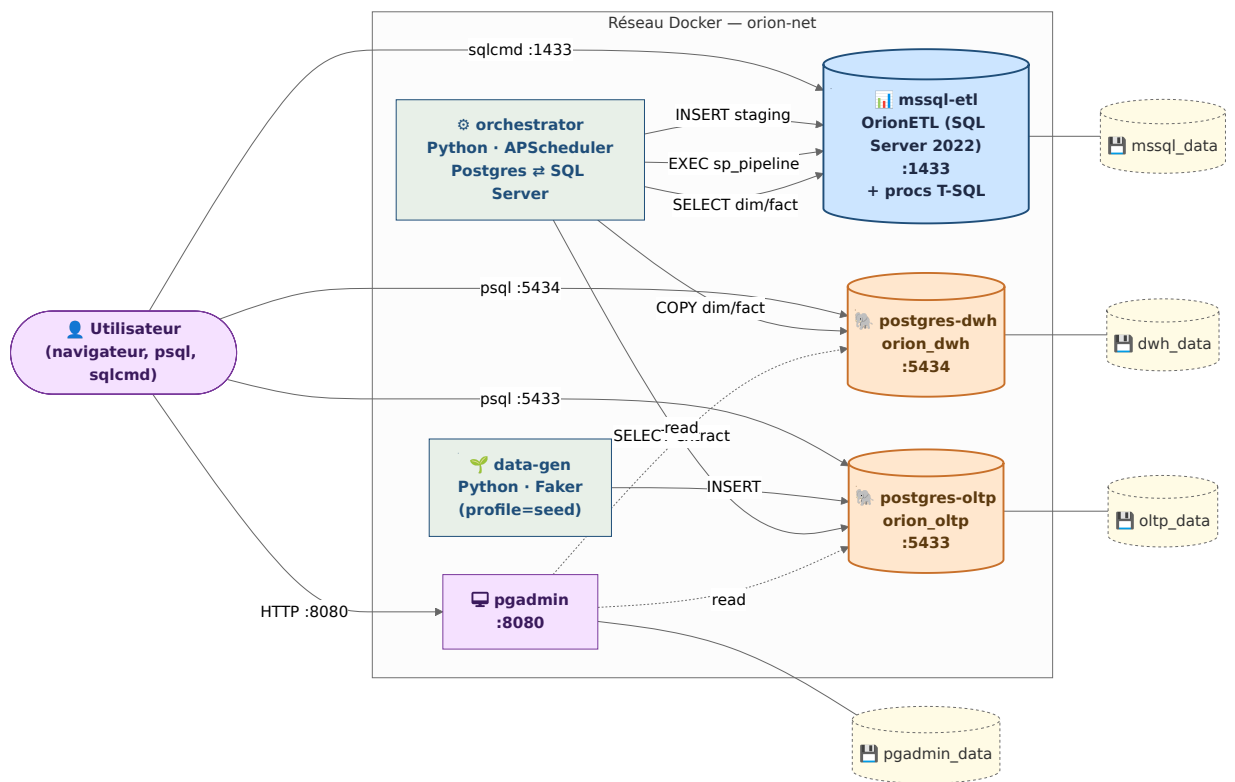


Figure 7 – Stack Docker : Postgres OLTP, SQL Server hébergeant la logique ETL, Postgres DWH, orchestrateur Python qui relie les trois, plus pgAdmin et data-gen.

- Capture d'écran à insérer

screenshots/docker-compose-ps.png

Sortie de `docker compose ps` : les six services à l'état Up (healthy).

Figure 8 – Sortie de `docker compose ps` : les six services à l'état *Up (healthy)*.

- Capture d'écran à insérer

screenshots/docker-startup.png

Suivi du démarrage : `docker compose up -d -build` puis `docker compose logs -f orchestrator` permettent de vérifier l'enchaînement `extract / EXEC sp_run_pipeline / push`.

Figure 9 – Suivi du démarrage : `docker compose up -d -build` puis `docker compose logs -f orchestrator` permettent de vérifier l'enchaînement `extract / EXEC sp_run_pipeline / push`.

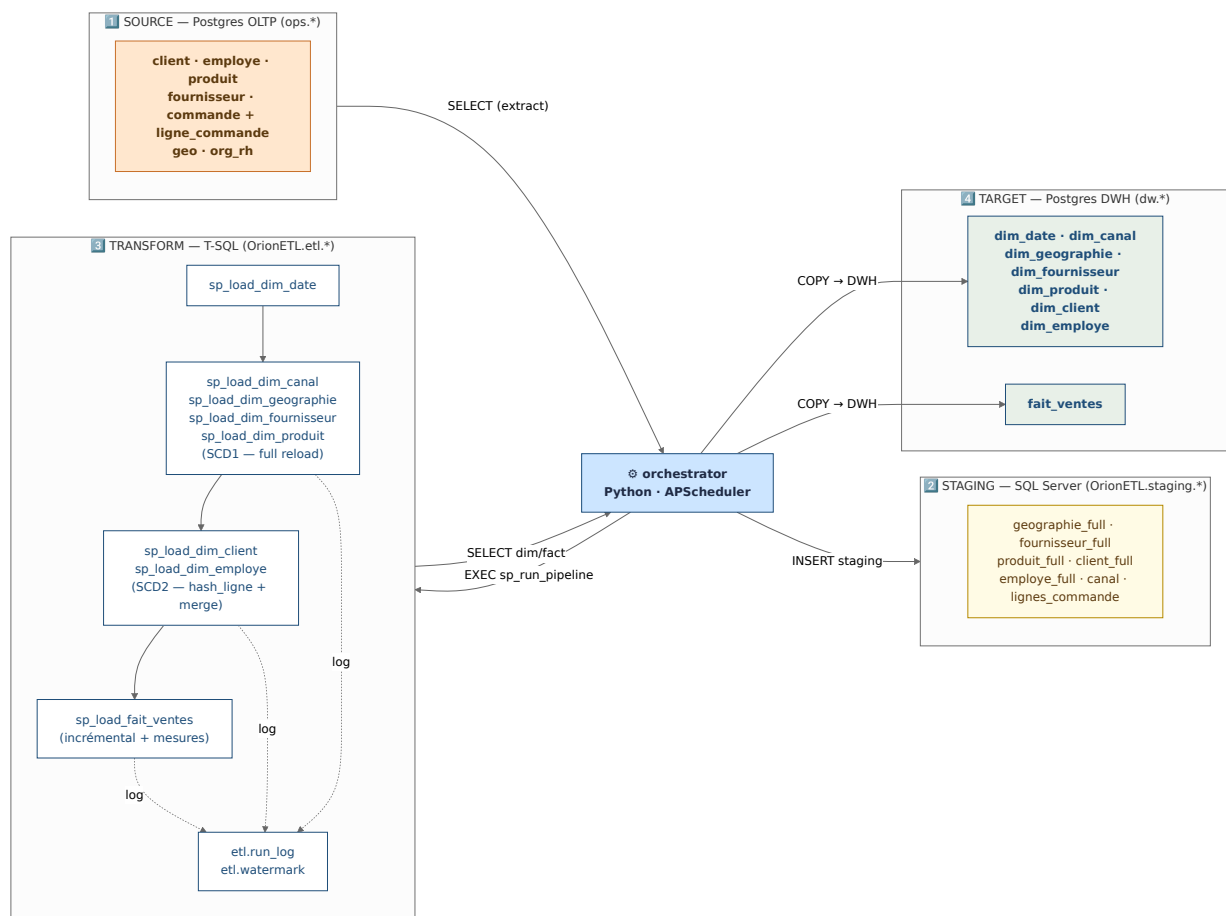


Figure 10 — Pipeline ETL en 4 étapes : 1. extract Postgres OLTP, 2. staging SQL Server, 3. transformation T-SQL (`etl.sp_run_pipeline`), 4. load Postgres DWH.

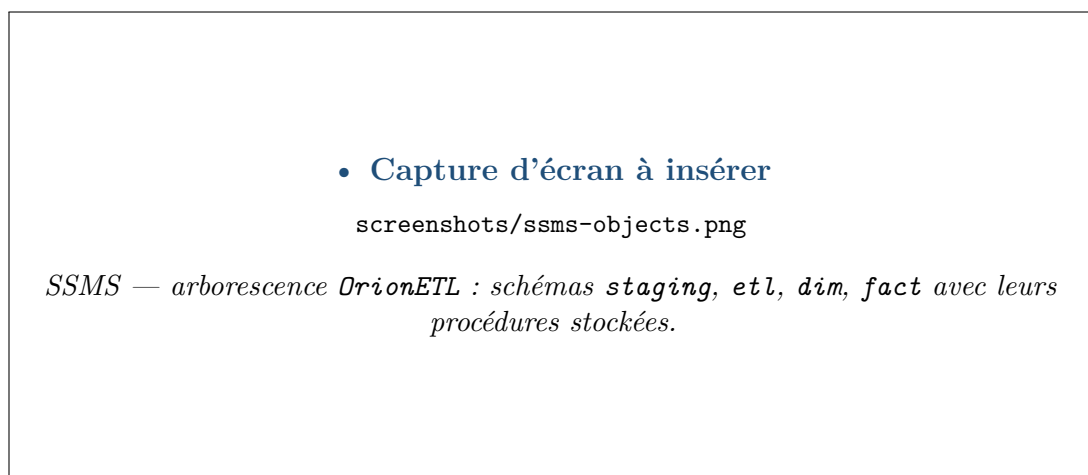


Figure 11 — SSMS — arborescence OrionETL : schémas `staging`, `etl`, `dim`, `fact` avec leurs procédures stockées.

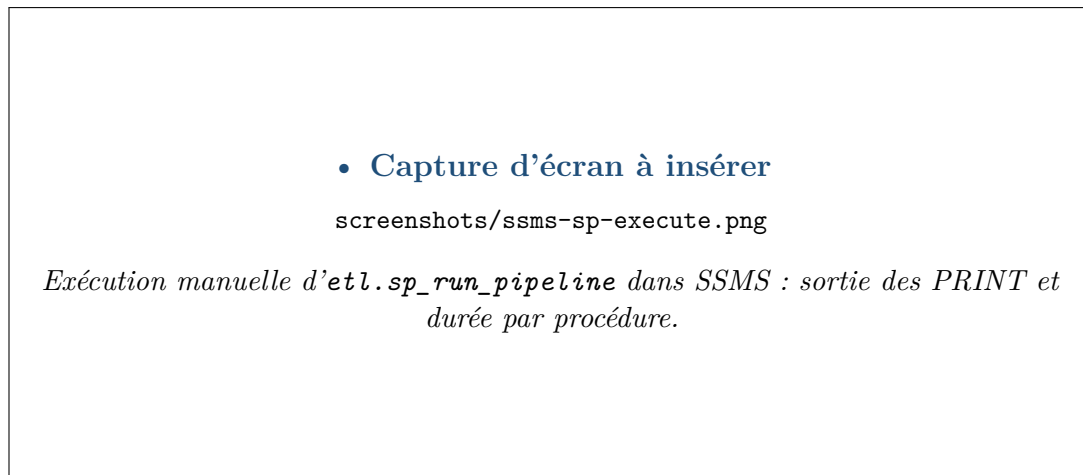


Figure 12 – Exécution manuelle d'`etl.sp_run_pipeline` dans SSMS : sortie des `PRINT` et durée par procédure.

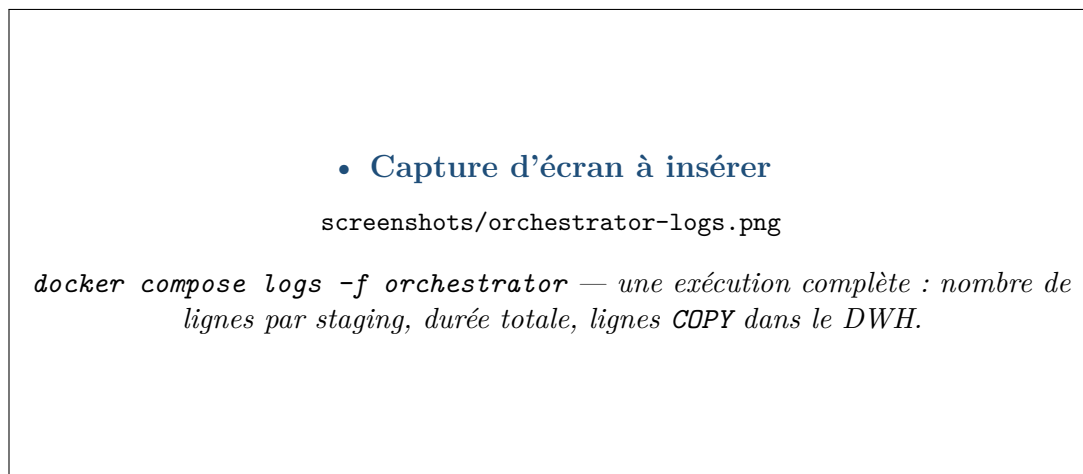


Figure 13 – `docker compose logs -f orchestrator` — une exécution complète : nombre de lignes par staging, durée totale, lignes `COPY` dans le DWH.

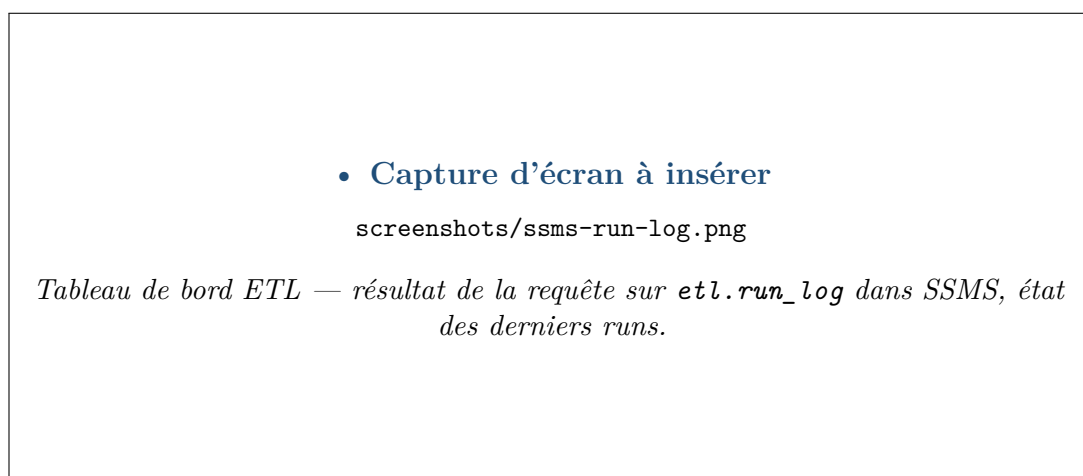


Figure 14 – Tableau de bord ETL — résultat de la requête sur `etl.run_log` dans SSMS, état des derniers runs.

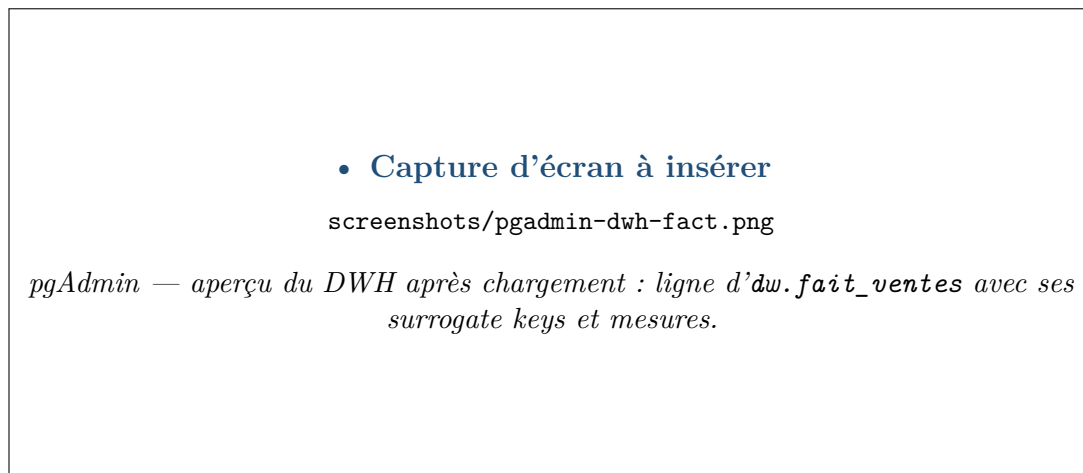


Figure 15 — pgAdmin — aperçu du DWH après chargement : ligne d'dw.fait_ventes avec ses surrogate keys et mesures.

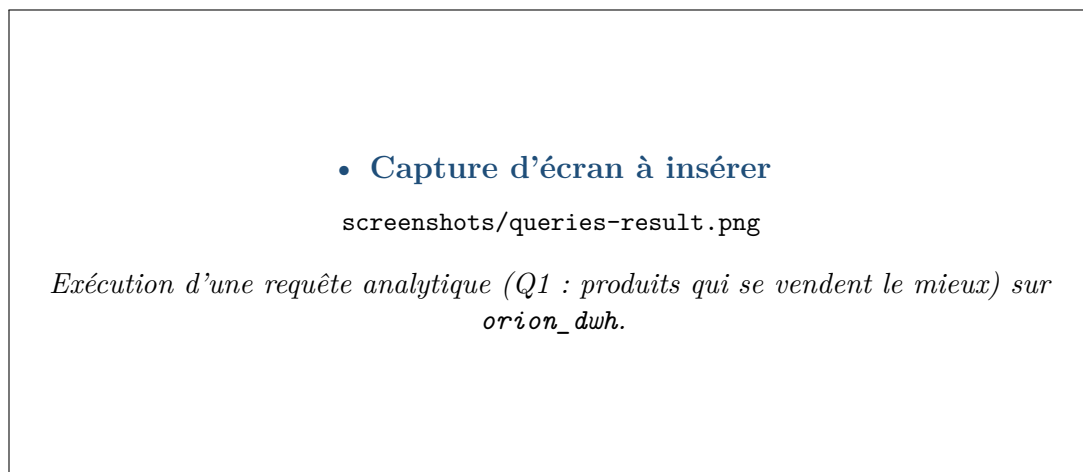


Figure 16 — Exécution d'une requête analytique (Q1 : produits qui se vendent le mieux) sur orion_dwh.